Password Hashing for Your Final Project

Supplementary handout (self-study) · optional project feature · uses the Lab 10 .env setup

> **Is this required?** No. Hashing is **optional** for the final project — you can get full marks for Database Security without it. But if your app has a login, hashing passwords is what every real system does, and it takes about 15 extra lines of code. If you do it, explain it in your demo.

1. Why never store plain passwords?

Imagine your `users` table looks like this, and someone gets a copy of it (a leaked backup, a stolen laptop, an SQL injection bug):

id	email	password	role
1	somchai@example.com	hello123	admin
2	nida@example.com	nida2005	user

The attacker now has every password — and because people reuse passwords, they can probably log in to your users' email and bank accounts too. The fix: store something the app can **check** a password against, but that **cannot be turned back** into the password. That is a **hash**.

2. Encoding vs encryption vs hashing

Technique	What it does	Reversible?	Use for passwords?
Encoding (e.g. Base64)	Rewrites data in another format. `hello123` → `aGVsbG8xMjM=`. Anyone can decode it.	Yes, by anyone	No — it hides nothing
Encryption (e.g. AES)	Scrambles data with a secret key. Whoever has the key gets the original back.	Yes, with the key	No — the key can leak too
Hashing (e.g. bcrypt)	Turns any input into a fixed-length fingerprint. There is no key and no "un-hash".	No	Yes

3. What a hash looks like

Here is SHA-256 (a common hash function) applied to two passwords that differ by one character. These were produced with Node's built-in `crypto` module:

```
const crypto = require('crypto');
crypto.createHash('sha256').update('hello123').digest('hex');
// 27cc6994fc1c01ce6659c6bddca9b69c4c6a9418065e612c69d110b3f7b11f8a
crypto.createHash('sha256').update('hello124').digest('hex');
// 0adab7ae55361051e894624d2e8f35438e7193f160c8de94e72190142cb02d12
```

Three properties to notice:

- **One-way** — you cannot get `hello123` back from the 64 hex characters.

- **Deterministic** — the same input always gives the same hash. That is how login works: hash what the user typed and compare.
- **Avalanche effect** — changing one character changes the whole hash, so similar passwords look nothing alike.

4. Why plain SHA-256 is not enough

SHA-256 is a good hash for files and data integrity, but it has two weaknesses for passwords:

1. **It is too fast.** On the laptop used to write this handout, Node computed **100,000 SHA-256 hashes in under 0.1 seconds**. An attacker with a GPU can try billions of guesses per second against a stolen table.
2. **Same password → same hash.** If two users both pick `hello123`, their rows are identical, and attackers have pre-computed tables ("rainbow tables") of hashes for millions of common passwords.

The fixes are a **salt** (a random value mixed into each password, stored with the hash, so equal passwords get different hashes) and a **slow, tunable** algorithm. **bcrypt** does both for you automatically, which is why we recommend it.

Same password, hashed twice with bcrypt — note they are different, yet both verify as `hello123`:

```
$2b$10$bdNtBA9Nz7.dcKj2Dl/orevmkT.TU0uvtdi0kjtkYuBE4d0gpcYGy
$2b$10$mQitw4fQX7bnMmjuE7o.qO3RUHVlJe2AxCZQXSbzHBUTw/Y0hJsYe
```

Part of the bcrypt string	Meaning
`$2b$`	Algorithm version (bcrypt).
`10$`	Cost factor: 2^10 = 1,024 rounds. Higher = slower = harder to crack. 10–12 is typical.
next 22 characters	The random salt — generated for you and stored inside the string.
last 31 characters	The actual hash. The whole string is always 60 characters.

> Because the salt and cost are stored inside the hash, you need **only one column** in MySQL. You never store the salt separately and you never store the password.

5. The users table

App users (people who log in to your website) live in an ordinary table — they are **not** MySQL accounts. Your app still connects to MySQL through one least-privilege account from `.env`, exactly as in Lab 10.

```
CREATE TABLE users (
  id            INT AUTO_INCREMENT PRIMARY KEY,
  email         VARCHAR(100) NOT NULL UNIQUE,
  password_hash VARCHAR(255) NOT NULL,      -- bcrypt needs 60; 255 leaves room
  role          ENUM('developer','admin','user') NOT NULL DEFAULT 'user',
  created_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

The `role` column is what your Node code checks for the Access Control criterion (developer / admin / user).

6. Hashing in Node.js with bcryptjs

Install the library in your project folder. We use **bcryptjs** (pure JavaScript) rather than `bcrypt` because it needs no C++ build tools, so it installs the same way on macOS Apple Silicon and Windows:

```
npm install bcryptjs
```

You only need two functions:

Function	Use it when…
`bcrypt.hash(password, 10)`	A user **registers** or **changes** their password. Returns the 60-character string to store.
`bcrypt.compare(password, hash)`	A user **logs in**. Returns `true` / `false`. It reads the salt and cost from the stored hash itself.

6.1 Register (controllers/authController.js)

```
const bcrypt = require('bcryptjs');
const pool   = require('../config/db');      // the .env-based pool from Lab 10

exports.register = async (req, res) => {
  const { email, password } = req.body;
  if (!email || !password || password.length < 8) {
    return res.status(400).json({ error: 'Need email + 8-char password' });
  }
  try {
    const hash = await bcrypt.hash(password, 10);          // hash BEFORE the INSERT
    await pool.execute(
      'INSERT INTO users (email, password_hash) VALUES (?, ?)',  // parameterised
      [email, hash]
    );
    res.status(201).json({ message: 'Account created' });
  } catch (err) {
    if (err.code === 'ER_DUP_ENTRY')
      return res.status(409).json({ error: 'Email already registered' });
    console.error(err);
    res.status(500).json({ error: 'Something went wrong' });  // generic message
  }
};
```

6.2 Log in

```
exports.login = async (req, res) => {
  const { email, password } = req.body;
  const [rows] = await pool.execute(
    'SELECT id, password_hash, role FROM users WHERE email = ?', [email]
  );
  const user = rows[0];
  const ok = user && await bcrypt.compare(password, user.password_hash);
  if (!ok) {
    // same message for 'no such email' and 'wrong password'
    return res.status(401).json({ error: 'Invalid email or password' });
  }
  res.json({ id: user.id, role: user.role });   // never send password_hash back
};
```

Then wire them up in a router, as in Lab 6: `router.post('/register', auth.register)` and `router.post('/login', auth.login)`.

6.3 Seeding test accounts

You cannot type a hash by hand into Workbench. Either register your test users through the app, or generate a hash once in the terminal and paste it into an `INSERT`:

```
node -e "require('bcryptjs').hash('admin1234', 10).then(console.log)"
```

7. Do and don't

Do	Don't
Hash in Node, then INSERT the hash with `?` placeholders.	Use MySQL's `SHA2()` / `MD5()` in the query — the plain password then travels in the SQL text and can end up in logs.
Use `bcrypt.compare()` to check a login.	Hash the login attempt yourself and compare strings — bcrypt salts differ every time, so that never matches.
Return the same error for unknown email and wrong password.	Say "email not found" — it tells attackers which accounts exist.
Keep DB credentials in `.env` (Lab 10).	Store the password, or Base64 of it, anywhere — not in a column, a log or a `console.log`.

8. Showing it in your final demo

1. Open the `users` table in Workbench: the `password_hash` column shows `$2b$10$…` strings, not passwords.
2. Register two users with the **same** password and show that their hashes are different (salt).
3. Log in with the right password (works) and a wrong one (generic error).
4. Show the two lines in your code that call `bcrypt.hash` and `bcrypt.compare`, and be ready to explain why plain SHA-256 is weaker.

> **Using SHA-256 instead?** Allowed, since hashing is optional — but expect a Q&A question about salts and speed (Section 4). bcrypt is the better answer.